Stress Test Framework for Viewport-Editor Actions

OpenCode (deepseek-v4-flash)

June 12, 2026

Abstract

The research, design, and writing of this work were conducted under the direction and supervision of Michael Conrad.

The viewport-editor plugin is an MCP (Model Context Protocol) server that provides windowed file editing capabilities to AI agents through 11 tools and approximately 35 distinct actions. Its buffer architecture — a single in-memory string replaced on every edit operation — creates a combinatorial correctness risk: sequences of edits at overlapping positions can produce cumulative corruption that no individual unit test can catch. We present a three-component stress test framework designed to exercise every exposed action in combination, using self-contained log directories for post-hoc analysis and a two-layer audit (mechanical + AI semantic) for zero-noise verification. The framework operates under a strict separation of concerns: generators produce sequences as data, the runner executes without assertions, and the audit layer evaluates correctness independently. A dense suite of 272 sequences runs in approximately 4 seconds, producing 37 mechanically-flagged findings that were all confirmed as correct tool behavior through sequential clean-room AI audit. No bugs were found.

1 Introduction

The viewport-editor plugin [1] exposes tools for file navigation, text editing, clipboard operations, diff application, regex testing, and search — all backed by an in-memory buffer where edits accumulate before being flushed to disk. The editing engine operates on a single-string replacement model: every call to `edit:replace`, `edit:delete-lines`, `edit:insert-lines`, or similar operations replaces the entire buffer content string. This is not an incremental line-based editor.

This architecture creates a class of defects that unit tests cannot easily catch: cumulative corruption from mixed-operation sequences. A unit test for `edit:replace` validates that a single replacement works. It does not validate that `edit:delete-lines` followed by `edit:insert-lines` at the same offset

produces the correct result, or that `edit:replace-all` followed by `edit:replace` correctly fails when the target text no longer exists. These are not edge cases — they are the normal operating pattern of an AI agent that performs multiple edits before saving.

The stress test framework described in this paper is designed to fill this gap. It systematically exercises all action combinations, logs results in self-contained directories, and uses a two-layer audit — mechanical checks followed by AI semantic analysis — to distinguish correct behavior from bugs.

2 Design Intent and Guiding Principles

The framework was designed around four hard constraints:

Separation of concerns. The framework is three independent components — generators, runner, and audit — each with a single responsibility. Generators produce sequences as data records. The runner executes sequences against the server and writes logs. The audit reads logs and evaluates correctness. No component depends on the internals of another. This allows the runner to be fast (no assertions), the audit to be thorough (slow, single-pass), and the generators to be modular (one per action category).

Zero assertions in the runner. The runner makes no claims about correctness. It only executes tool calls and records responses. This is deliberate: separating execution from verification means the runner can be trusted as a neutral recorder. Any assertion in the runner would introduce the possibility that the assertion itself is wrong, contaminating the evidence.

Self-contained logs. Each sequence produces its own directory with the seed content, parameters, ordered step results, and final disk state. The audit can inspect any single sequence without cross-referencing the manifest or other sequences. This matters for AI semantic analysis: a clean-room sub-agent must receive exactly one entry's data — no more, no less.

Zero-noise semantic audit. Anomalies found by the audit must be either confirmed bugs or framework defects. False positives are not acceptable. If the semantic auditor returns ANOMALY, it must be a real semantic defect or tool misbehavior — not a misunderstanding of the behavioral invariant that errors leave the buffer in a valid prior state.

3 Approaches Discarded

Several approaches were considered and rejected before arriving at the final design.

3.1 Inline Assertions in Runner

The simplest approach is to embed assertions directly in the runner: after each sequence, check expected content against actual content and report PASS/-FAIL. This was rejected because it conflates execution with verification. A false positive in an assertion (asserting the wrong expected value) would produce a misleading result indistinguishable from a correct test passing. Worse, a runner crash during a sequence would lose all prior results for that run. Separation ensures that log files survive even if the audit encounters errors.

3.2 Behavioral RED/GREEN per Sequence

Following the test-driven development pattern, each sequence could be written as a behavioral RED test (fails against current code) then GREEN (passes after a fix). This was rejected because the stress test is not a development tool — it is an investigative tool. We do not know in advance what the correct behavior should be for every mixed sequence; discovering that is the purpose of the exercise. Writing a RED test presumes knowledge of the expected outcome.

3.3 Git-Style Three-Way Comparison

An alternative approach is to model each sequence as a diff: apply the same operations to a reference implementation and compare outcomes. This was rejected because no reference implementation exists. The viewport-editor plugin is the only implementation. A reference would be a reimplementation, introducing the possibility that the reference contains the same bugs or different bugs.

3.4 AI-Agent-Initiated Testing

The initial proposal was to have an AI agent generate and execute test cases dynamically using `opencode-cli run`. This was rejected for multiple reasons: AI agents produce non-deterministic output, making reproduction impossible; the cost of dispatching an LLM inference per test case is orders of magnitude higher than an in-memory MCP call; and the agent's reasoning about what constitutes a correct result can be contaminated by its own training data.

3.5 Parallel Sub-Agent Dispatch

For the AI semantic audit phase, sub-agents could be dispatched concurrently to reduce wall-clock time. This was rejected because each sub-agent must be clean-room — receiving no context about other findings, prior results, or orchestrator expectations. Concurrent dispatch risks cross-contamination through shared infrastructure. Sequential dispatch is slower but guarantees isolation.

4 The Framework

The framework consists of three components, described below.

4.1 Generators

Each generator module exports a single function returning a list of `Sequence` objects. The `Sequence` dataclass carries the seed content, ordered tool calls, expected save behavior, and content checks.

Five generator modules exist:

- **Edit pairs** — all ordered pairs of the six edit operations (`replace`, `replace-all`, `insert-lines`, `delete-lines`, `swap-lines`, `move-lines`) at non-overlapping, overlapping, and same positions, in single-line and multi-line content. Total: 216 pair sequences in dense mode, plus 6 triple-chain high-risk sequences. The triple chains test patterns like `delete-lines` → `insert-lines` → `delete-lines` where offset correctness accumulates across three operations.
- **Clipboard** — 8 sequences covering copy/cut/paste lifecycle, stash/pop/swap semantics, and error paths (paste with empty clipboard, pop nonexistent stash slot).
- **Diff apply** — 7 sequences covering single-hunk apply, multi-hunk offset tracking, fuzzy context matching, boundary hunks, overlapping hunks (error path), and apply on already-modified buffer.
- **Crosstalk** — 2 sequences covering multi-viewport open of the same file and open/close/reopen lifecycle.
- **Mtime cross-session** — 3 sequences covering external file modification detection: stale mtime blocks save, force flag overrides, and buffer refresh on reopen.

4.2 Runner

The runner connects to an in-memory FastMCP server using the same `create_server` pattern as the project's test fixtures. For each sequence, it:

1. Creates the seed file on disk
2. Opens a viewport on the seed file
3. Executes the sequence steps in order, recording each tool call's response, error state, and duration
4. Attempts a `file:save` (unless the sequence expects save failure)
5. Reads the final disk state

6. Writes the sequence directory with `params.json`, `seed_content.txt`, `steps.json`, and `final_content.txt`

7. Closes the viewport

The runner is a bare loop with no exception handling beyond catching crashes per-sequence. A crash in one sequence does not abort the suite — the error is logged and execution continues. This is critical for dense suites where a single server-side crash should not invalidate the entire run.

4.3 Mechanical Audit

The mechanical audit runs post-hoc, requiring only the log directory. It performs four check categories:

1. **Hard errors:** Any step with `isError=true` that does not match an expected error pattern (e.g., "Edit target not found" after a prior `replace-all` consumed the target, or "invalid line range" after a prior `delete-lines` reduced the file length).

2. **Content checks:** Expected content markers (e.g., "REPLACED" after a `replace` operation) verified against the final content.

3. **Truncation:** If final content is significantly shorter than seed content with no deletions or cuts in the sequence.

4. **Seed line integrity:** Seed-origin lines that are absent from final content without being intentionally modified by a `replace` operation.

Sequences that pass all mechanical checks are marked PASS. Sequences that fail are emitted as entries in the findings manifest for AI semantic analysis.

4.4 AI Semantic Audit

The AI semantic audit is the second verification layer. It is not run by the audit script — the audit script produces a structured findings manifest. An orchestrator (an AI agent or human) dispatches clean-room sub-agents one per finding, sequentially.

Each sub-agent receives an invariant prompt containing:

1. Tool semantics reference (describing each tool's behavior and error conditions)

2. Critical behavioral invariant (explaining that error operations leave the buffer in a valid prior state and that saving after an error is not anomalous)

3. The specific finding's seed content, step summary with success/error annotations, and final content

The sub-agent must return either PASS (content is semantically correct) or ANOMALY (unexpected behavior that may indicate a bug). The prompt template is frozen — only three fields vary between findings. No orchestrator commentary, pattern labels, expectations, or prior results are included.

5 Validation Results

The dense suite executes 272 sequences in approximately 4 seconds against an in-memory MCP server. The mechanical audit flags 37 sequences (14% of the dense suite). All 37 flagged sequences were dispatched to clean-room AI sub-agents for semantic evaluation. All 37 returned PASS.

The 37 findings fall into three mechanical categories:

Edit target not found (19 findings). These occur when a prior operation consumes the target text for a subsequent operation. For example, `edit:replace-all("line" → "ROW")` changes all occurrences of "line" to "ROW", so a subsequent `edit:replace("line 4" → ...)` correctly fails because the string "line 4" no longer exists. Every such failure is correct tool behavior.

Invalid line range (12 findings). These occur when a prior `delete-lines` operation reduces the file length, making a subsequent operation's line range out of bounds. For example, deleting lines 2–4 from a 5-line file leaves 2 lines; attempting `delete-lines(3-5)` on a 2-line file correctly produces "invalid line range: 3–5 (total 2)".

Expected error path (6 findings). These cover clipboard operations on nonexistent stash slots and other intentionally invalid operations. All produce the expected error messages.

5.1 Dense vs. Sparse

The framework supports two density modes. The sparse suite (104 sequences) runs in approximately 1.6 seconds and is suitable for regression testing in CI. The dense suite (272 sequences) adds overlapping-position variants and multi-line content variants, making it suitable for independent stress testing.

6 Lessons Learned

Several design decisions proved critical to the framework's effectiveness:

The invariant prompt template is essential. Early attempts at semantic audit revealed that sub-agents produce false positives when the prompt varies between findings. A frozen template with only three variable fields eliminates

this source of noise. Any orchestrator that deviates from the template produces contaminated results.

The error invariant must be explicitly stated. Sub-agents consistently flagged "save after error" as anomalous — they interpreted the error as corrupting the buffer state. The critical behavioral invariant ("ERROR leaves buffer in prior valid state; save after ERROR is NOT anomalous") eliminated this entire class of false positives.

Sequential dispatch is non-negotiable. Parallel or batched dispatch of semantic sub-agents would introduce cross-contamination risk. Each sub-agent must receive exactly one entry's data and nothing else. This constraint makes the audit slower (wall-clock time proportional to number of findings), but the correctness guarantee justifies the cost.

Prompt contamination from the orchestrator is insidious. The single false positive in this study was traced to the orchestrator supplying incorrect seed content metadata ("10 lines" instead of "5 lines") in the prompt — not a bug in the sub-agent's reasoning or the plugin's behavior. This demonstrates that the orchestrator itself must be governed by the same zero-contamination protocol it enforces for sub-agents.

7 Conclusion

The stress test framework validates that the viewport-editor plugin handles all tested action combinations correctly. No bugs were found across 272 dense sequences covering edit pairs, clipboard operations, diff application, cross-viewport interaction, and mtime conflict detection. The framework's three-component architecture — generators, runner, audit — successfully separates concerns and produces verifiable, self-contained evidence. The two-layer audit (mechanical + AI semantic) provides a reliable mechanism for distinguishing correct behavior from bugs with zero permissible noise.

Future work includes adding sequences for the composite tools (`read_file`, `write_file`, `edit_text`, `find_text`) and extending the cross-session test coverage with real session boundary simulation.

References

[1] Michael Conrad. Viewport-editor: Windowed viewport editor mcp server for ai agents, 2026.